

Standing up on-device Small Language Models with IT — an enablement runbook

Audience: IT, Security, and QA/Validation. **Scope:** deploying a small open-weight language model **on a single workstation, fully offline**, paired with SAS/R, for the **language layer only** (classify · extract · route · draft short grounded text). **Out of scope:** any use where the model produces a number or a record — those stay with SAS/R and the validated engines (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS).

Lead with the paradox

On **data-governance grounds this is the easiest AI option you will ever review**. The model is offline on one box, so there is **no egress to firewall, no vendor BAA to chase, no telemetry to disprove* — *you can demonstrate the security control by pulling the network cable*. *The hardest** honest constraint is the opposite of the usual one: not "can we secure it" but "**is the small model actually good enough**" — which is why **scope is everything**. Do not let the easy security story tempt anyone into a hard language task. SAS/R and the validated tools own every number and record; the model is a constrained, offline helper behind a human gate. **Nothing here asks IT to trust an AI with a regulated value.**

Part 1 — the enablement playbook

1. **Pick the use cases and pin the language task to a schema.** Choose only short, bounded language sub-tasks — classification, field extraction, routing/triage tags, near-duplicate grouping, or a short RAG-grounded draft. For each, write down the exact sub-task and the **output schema** (the allowed labels/enum, the fields, the JSON shape). If you cannot reduce it to a **fixed-vocabulary, short-context** decision, it is beyond a small model — leave it deterministic SAS/R or escalate to a larger model. Do not force it on-device.
2. **Size the hardware — CPU-first.** A modern workstation runs a 7–8B model quantized to Q4 (e.g. Q4_K_M GGUF) at a usable, non-interactive speed **on CPU alone**; 3–4B models are comfortably faster. Memory rule of thumb: at Q4 the weights are ~0.5 GB per billion parameters (~4–5 GB for an 8B), but you must add the KV cache, the context, and runtime overhead — **budget ~6 GB resident for an 8B Q4 model and provision 16 GB+ system RAM** to run it alongside SAS/R. Add **one consumer GPU only where throughput demands it**; for nightly/batch triage, CPU is usually enough. A single 12–16 GB GPU fits an 8B model in VRAM and multiplies tokens/sec if a queue justifies it. (Note: CPU→GPU can change exact byte-for-byte outputs — see step 5.)

3. **Choose a runtime and install it in the locked-down environment.** Ollama for the simplest operations; **llama.cpp** for maximum control or to embed the engine (Ollama wraps llama.cpp — same GGUF runtime); **LM Studio** for desktop trials; **vLLM** only later if you genuinely need server-grade GPU throughput. Install from a ****signed, version-pinned release**** mirrored on the internal artifact repository (or an approved container image). ****Pull the model on a connected staging box, checksum it (SHA-256), then transfer the verified file to the offline workstation**** — the production box never reaches the internet.
 4. **Wire SAS/R to the local endpoint and bolt on the validator + human gate.** SAS calls the loopback endpoint with `PROC HTTP`; R uses `httr2` — both target **`127.0.0.1` only**. SAS/R does all data access and computation, then sends just the short language sub-task. The model returns **schema-constrained JSON** (enforced with the runtime's grammar/JSON-schema mode, not merely *asked for* in the prompt); a **SAS/R validator** parses it and checks every field against the allowed values, **rejecting and logging anything malformed or off-list**. Nothing reaches a record of truth until a human reviews and approves it. ***(The accompanying LOCALMIND helper library implements exactly this — see Part 4.)***
 5. ****Pin the configuration for best-effort reproducibility; register it; do not claim bit-exactness.** **Freeze** model name + quantization + SHA-256 digest******, the decoding settings (temperature 0 / greedy, `top_k/top_p` pinned, penalties pinned, a fixed seed), and the full **runtime context** (runtime build/version, hardware, thread count, single-stream `batch = 1`). Be honest with QA: temperature 0 makes token selection deterministic, but it does **not** guarantee byte-identical output across different hardware or under varying load, because floating-point addition is non-associative. Pinning the runtime, hardware, and `batch = 1` (CPU single-stream is the most reproducible path) gets you ****run-to-run stable output on that box; treat cross-hardware bit-exactness as not assured.**** Enter the frozen tuple in the **engine-of-record / model-freeze registry** exactly as a version of a qualified tool — no silent model, quant, runtime, or hardware swaps.
 6. **Validate (GAMP 5 Second Edition, risk-based) and wire the Part 11 audit trail.** Qualify the configured system as **decision-support**, not as a record generator: IQ the install, **OQ the pinned model on a fixed test set** (include a re-run stability check), and PQ on the real workflow. The 21 CFR Part 11 audit trail is the real backbone — ****SAS/R logs, per item, the prompt, the model name + quant + digest, the model's output, and the human approve/edit/reject decision**** — so every AI-assisted action is attributable and reconstructable, and the frozen model makes it re-runnable for an inspector.
 7. **Define the support model and phase the rollout.** Name an **owner** for the runtime and the model registry, set a **documented patch cadence**, and put any change to the model, quantization, runtime build, or decoding/hardware config under ****change control with re-validation**** — exactly like a version bump on a qualified analytical tool. Roll out in phases: one **Ready** (green) use case on one workstation → measure → widen to the **Guardrails** (amber) set → only then consider a GPU host if a real queue justifies it.
-

Part 2 — anticipated hurdles, and how to clear each

Hurdle (what they'll raise)	Why it comes up	How to clear it
"We don't allow unknown binaries or downloads."	Hardened endpoints block arbitrary executables and internet pulls; the runtime and model are both new artifacts.	Use a signed, version-pinned release from the internal artifact mirror or an approved container image. Air-gap the model transfer: pull on a staging box, checksum (SHA-256), move the verified file to the offline box. Nothing is pulled live on production.
"We have no GPU budget."	People assume "AI" means a GPU farm.	It doesn't here. CPU-first: a small quantized model (3-8B, Q4) runs at usable speed on a normal workstation with ~6 GB resident and 16 GB+ system RAM. Add a single consumer GPU only on the specific component where throughput actually demands it.
"Is the AI validated? Is it deterministic?"	QA cannot accept a non-reproducible component in a GxP path.	Be precise, not promotional. Frozen model (name+quant+digest) + temperature 0 + pinned runtime/hardware + batch = 1 gives run-to-run stable output on the box, but byte-exactness is not guaranteed across hardware (floating-point non-associativity). So the controls are: GAMP 5 2nd ed. risk-based CSV (IQ/OQ/PQ with a re-run stability check) qualifying it as decision-support ; a SAS/R validator on every output; a human gate before any record; and a Part 11 audit line capturing the actual output and the human decision. The SLM is never the record — SAS/R and the validated tools are.
"Where does the data go?"	The default fear with any AI is data leaving to a vendor.	Nowhere. The model is offline on the device; SAS/R only ever calls 127.0.0.1 and asserts the loopback before use. Demonstrate it by pulling the network cable — the workflow still runs. This is the simplest data-governance story of any AI option: no egress, no vendor, no telemetry.
"What about endpoint / device security?"	A workstation now holds study text and runs an inference service.	Treat the box as a controlled instrument: full-disk encryption, standard endpoint management, least-privilege access, the inference service bound to loopback only . An offline box has no outbound channel to exfiltrate through; harden it like any other instrument holding study data.
"What about the open-weight model's licence?"	Legal must confirm commercial use and provenance before it ships, and open-weight licences vary by size within the same family.	Prefer genuinely permissive weights — IBM Granite 3.x (Apache-2.0) or Phi-3.5/Phi-4-mini (MIT) . Watch the traps: Qwen2.5-7B is Apache-2.0 but Qwen2.5-3B is the custom Qwen licence , and Gemma 2 is under Google's "Gemma Terms of Use," not Apache/MIT. Legal reviews the specific model and size variant , its licence, and its provenance — recorded with the registry entry, not waved through.
"Who supports and patches it?"	Ungoverned shadow tooling is an audit	A named owner , a documented patch

Hurdle (what they'll raise)	Why it comes up	How to clear it
	finding.	cadence for the runtime, and the model under a change-controlled freeze registry . Any change to the model, quantization, runtime build, or decoding/hardware config is a controlled change requiring re-validation.
"Small models hallucinate — how do we trust the output?"	Correct instinct; SLMs hallucinate more and are more prompt/format-sensitive than frontier models.	The risk is bounded by design : grammar/schema-constrained output + RAG grounding on a short retrieved passage + a SAS/R validator that rejects anything off the allowlist + a human gate. And critically, the model never produces a number — all counts, checks, reconciliation, and stats are SAS/R's and the validated engines'.
"Will it keep up at volume?"	A small model on CPU has finite throughput.	Batch and schedule off-hours (these jobs are nightly, not interactive), right-size the model (the smallest that passes OQ), and only then scale to a single GPU host if a real queue justifies it — re-validating, since a hardware change can shift exact outputs.

Choosing the model (early 2026 — verify current versions and the exact size's licence)

The model is the **least important** decision — the scaffolding (constrained output, RAG, the SAS/R validator, the human gate) is what makes a small model usable, and it is the same whichever model you pick. Choose one that is **licence-clean, well-supported, and small enough to validate, **then** prove it on a fixed test set for your specific sub-task. ** Capability is established by your validation run, not by the model card.

Model family	On-device sizes	License & commercial-use note	Good for
Llama 3.1 / 3.2 (Meta)	8B; 3B, 1B	Llama Community License — custom, not OSI-approved. Commercial use permitted, but with an Acceptable Use Policy, a "Built with Llama" attribution requirement, and a restriction on using outputs to train non-Llama models. Legal must read the full terms, not just the >700M-MAU clause.	Strong, widely-supported default. 8B for classify/extract/short drafting; 3B/1B for high-volume tagging on lighter hardware.
Qwen2.5 (Alibaba)	7B, 3B, 1.5B, 0.5B	Apache-2.0 for 0.5B/1.5B/7B ; the 3B variant is the custom Qwen Research License. Verify the licence on the exact size; prefer an Apache-2.0 size.	Good instruction-following and structured-output adherence; the 1.5B/0.5B Apache sizes suit cheap routing/triage.
Phi-3.5-mini / Phi-4-mini (Microsoft)	~3.8B	MIT — the most permissive here, minimal review burden.	Competitive among ~4B models on bounded extraction and short structured tasks; validate its

Model family	On-device sizes	License & commercial-use note	Good for
			reasoning on your task.
Gemma 2 (Google)	9B, 2B	Gemma Terms of Use — custom, not OSI-approved; binding Prohibited Use Policy passed to downstream users and derivatives. Usable, but legal must read it. (Specific to Gemma 2; later releases changed licensing.)	9B is among the stronger small models for grounded drafting; 2B for lightweight tagging.
IBM Granite 3.x (IBM)	8B, 2B	Apache-2.0 — clean, OSI-approved, explicitly built for enterprise/structured tasks.	Designed for exactly this stack's work: classification, extraction, RAG, JSON output. A sensible enterprise-first default.

How to choose: prefer **Apache-2.0 / MIT** (Granite, Phi, Apache sizes of Qwen2.5); pick the **smallest model that passes your validation** on the task (a 3B that passes beats an 8B you never validated); use the **instruct** build and confirm it follows grammar/JSON-constrained output. If none clears the bar, that is a signal the task is **beyond** on-device SLM — not a prompt to ship a failing model.

Part 4 — the SAS/R glue, in one minute (LOCALMIND)

The companion LOCALMIND library (`slm_macros.sas + slm_companion.R`) makes the wiring a few lines and enforces the five controls so IT/QA don't have to take them on faith:

1. **Offline** — `%slm_init / slm_init()` refuse any non-loopback endpoint and probe the local server; the box stays offline.
2. **Schema-constrained** — every call sends a full JSON Schema so the model can only emit allowed tokens.
3. **Validated** — `%slm_validate` checks every field against a hard-coded allowlist and range; off-list ⇒ reject and **fail loud**, never coerce.
4. **Pinned & frozen** — model tag + quantization + digest + temperature 0 + seed, stamped onto every result.
5. **Human-gated** — drafts are advisory; SAS/R writes only after approval and emits the Part 11 audit line. **The model never produces a number.**

The honest bottom line for IT

The on-device SLM is the **easiest AI option to approve on data-egress and privacy grounds** — nothing leaves the box, and you can prove it physically. The **hardest** honest constraints are two: **model capability** (a small quantized model is a narrow language helper, not a reasoning

engine) and the fact that "**deterministic**" is a **configuration-pinned, best-effort property**, not a guarantee of byte-exact reproduction across hardware. Both are handled by the same discipline: keep the language work **short, bounded, schema-constrained, and validated**; pin model + quant + runtime + hardware + decoding and **register it**; capture the actual output and the human decision in the **Part 11 audit trail**; leave every number to SAS/R and the validated engines; and send anything needing real synthesis or nuance to a larger model or keep it deterministic. **Scope and pin honestly, and this is a clean, inspectable, sovereign AI assist.**